atlas

TECHNISCHES WHITEPAPER · AUSGABE 1.0

Applikationen. Von der Idee in den Betrieb.

Architektur, Entwicklung, Releases und Verteilung von Prozessapplikationen – zwischen implementiertem Stand und bewusst offenem Zielbild.

FAKTENBASIS

6. November 2026

EINORDNUNG

Architektur und Implementierungsstatus

FORMAT

Fachpublikation · Deutsch (Schweiz)

Leseschlüssel. Umgesetzt bedeutet: durch ADR-Index und konkrete Implementierung oder Tests belegt. **Teilweise umgesetzt** bedeutet: Teile des beschlossenen Ansatzes sind vorhanden, der Gesamtumfang nicht. **Geplant/offen** bezeichnet nicht implementierte, noch unentschiedene oder ausdrücklich vorgeschlagene Teile. **Einschätzung** kennzeichnet eine fachliche Bewertung bzw. Empfehlung und keine Produkteigenschaft. Ein Whitepaper ist keine unabhängige Sicherheitszertifizierung und kein Abnahmenachweis.

Inhaltsverzeichnis

1. Management Summary
2. Ausgangslage und Problem
3. Begriffe und Abgrenzung
4. Architekturüberblick
5. Bestandteile einer Applikation
6. Lebenszyklus
7. Entwicklung einer Applikation
8. Versionierung und Migration
9. Konfiguration und Konstanten
10. Verteilung
11. Sicherheit und Berechtigungen
12. Betrieb
13. Durchgehendes Fallbeispiel
14. Offene Punkte, Grenzen und Ausblick
15. Glossar und ADR-Verzeichnis

1. Management Summary

Eine **atlas-Applikation** fasst zusammengehörige BPMN-Prozesse und weitere Entwurfsartefakte in einer benannten, versionierbaren und transportierbaren Einheit zusammen. Sie ist mehr als ein Verzeichnis: Der fachliche und technische Stand wird beim Publizieren als **Applikations-Release** dokumentiert. Eine einzelne Prozessdefinition behält daneben ihre eigenständige Versionsgeschichte. Diese Trennung ist notwendig, um ein fachliches Angebot als Ganzes auszuliefern und dennoch einzelne laufende Prozessinstanzen mit ihren gebundenen Definitionen zu betreiben. ADR-0034, ADR-0128.

Umgesetzt: Der Applikationscontainer, lokale Publikation und Release-Historie, Deployment-Ziele mit Promotion, Quellbaum-Export und -Import, Paketimport mit Vorschau und Konfliktentscheidungen sowie Konstanten je Installation sind vorhanden. Die Applikationspakete können auch Produktkataloge und weitere Architekturinformationen enthalten. Die Umsetzung wird durch API-Handler und Tests gestützt, unter anderem `api/promote.go`, `api/apppackage.go`, `api/constantsimport.go` und ADR-0528.

Teilweise umgesetzt: ADR-0128 ist insgesamt **Partial**, weil die direkte Git-Anbindung fehlt. atlas hub bietet den signierten Registry-Vertrag, eine fest konfigurierte Quelle, geschützten Download und einen überprüfbaren Importpfad, aber noch keinen vollständigen Lifecycle für mehrere Quellen, durchgängige Installations-Provenienz und verwaltete Updates. Die Architektur-Snapshots von Releases sind implementiert, während zusätzliche Darstellungsschichten und Confluence-Publikation nicht vollständig sind. ADR-0128, ADR-0134, ADR-0499, ADR-0471.

Geplant/offen: Ein allgemein verbindlicher Referenzweg von Entwicklung über Test und Integration zur Produktion ist noch kein verabschiedeter Gesamtstandard. Die in diesem Whitepaper beschriebene Freigabekette ist ein **Zielbild**, keine bereits durch die Plattform durchgesetzte Vier-Stufen-Pipeline. Der Lehrgang Prozessentwicklung nennt Umgebungsgrenzen, Freigaben und Schreibschutz in Produktion ausdrücklich als offene Entscheidungen.

Einschätzung: Für einen gegenwärtig belegbaren Produktionsprozess ist die manuell kontrollierte, protokollierte Promotion eines eingefrorenen Applikations-Releases der konservativere Startpunkt, sofern die Netz- und Berechtigungsgrenzen passen. atlas hub ist für unabhängige, signierte Pull-Verteilung eine starke Alternative; die heutige Ein-Registry- und Netzrestriktion verhindert jedoch bestimmte interne Topologien. Beide Verfahren verlangen ergänzende organisatorische Freigaben, Umgebungsinventare und Betriebsprüfungen. Das gilt besonders dann, wenn produktive Zielsysteme oder Personendaten betroffen sind.

---

2. Ausgangslage und Problem

Ein BPMN-Diagramm beschreibt einen Ablauf. Es beschreibt aber noch nicht die vollständige fachliche Anwendung: Ein Benutzer benötigt Formulare und Startmöglichkeiten; Entscheidungen benötigen DMN-Modelle; Service Tasks benötigen konfigurierte Worker; Umgebungen benötigen unterschiedliche Identitäten, Ziele und Konstanten. Auch Produktkataloge, Informationsmodelle und die Zuordnung zu Geschäftsfähigkeiten können zusammengehören. Eine lose Sammlung von Dateien kann diese Beziehungen nicht verlässlich für eine Freigabe, Installation oder Weiterentwicklung transportieren. ADR-0034, ADR-0467, ADR-0528.

Die ursprüngliche Entwurfsoberfläche kannte ungruppierte BPMN-Entwürfe und separat versionierte Deployments. ADR-0034 führte Projekte als Design-Time-Container ein. ADR-0128 hob diese Container zur **Prozessapplikation** an: mit dem fachlich sichtbaren Publikationsvorgang, einem Release-Manifest und einer Geschichte, die nachvollziehbar macht, welche Prozesse zusammen freigegeben wurden. Der ältere interne Speichername `projectId` darf weiterexistieren; die Oberfläche und öffentlichen Applikationsschnittstellen müssen deshalb nicht unmittelbar dem historischen Speichernamen folgen.

Drei Fragen sind zu trennen: **Was bearbeiten wir?** Die Entwürfe und Modelle. **Was haben wir veröffentlicht?** Das eingefrorene Release mit seinen Versionen. **Wo wird es betrieben?** Die konkrete atlas-Installation mit ihren eigenen Identitäten, Worker-Konfigurationen und laufenden Instanzen. Bei einem Umgebungswechsel ändert sich nicht automatisch alles zugleich. Genau darin liegt die Notwendigkeit des Applikationskonzepts.

---

3. Begriffe und Abgrenzung

Begriff	Bedeutung und Grenze
Projekt	Historischer Name des Design-Time-Containers aus ADR-0034; in der heutigen Fachoberfläche als Applikation geführt.
Applikation	Benannte Einheit zusammengehöriger Entwurfsartefakte, lokaler Releases und gegebenenfalls Katalogbeziehungen; **keine** Prozessinstanz.
Artefakt	BPMN-Entwurf, Entscheidungsreferenz, DMN-Modell, Formular, Informationsmodell oder weiteres zugeordnetes Entwurfsobjekt.
Prozessdefinition	Validiertes, kompiliertes und versioniertes ausführbares BPMN-Modell.
Prozessinstanz	Konkrete laufende bzw. abgeschlossene Ausführung einer Prozessdefinition mit eigenem Zustand und Ereignissen.
Applikations-Release	Lokal versioniertes Manifest der zusammen publizierten Artefakte und deren konkreten Prozessversionen.
Deployment	Registrierung ausführbarer Definitionen auf einer atlas-Installation; getrennt vom blossen Speichern eines Entwurfs.
Deployment-Ziel	Konfigurierter anderer atlas-Server, an den ein vorhandenes Release übertragen werden kann.

Begriff	Bedeutung und Grenze
Promotion	Push eines vorhandenen, eingefrorenen Releases zu einem solchen Ziel.
Paket	Export- und Importformat mit Quellbaum, gegebenenfalls Konstanten, Katalogen und installationsspezifischen Fragen.
Blueprint	Über atlas hub vertriebenes, signiertes und adressiertes Release eines Applikationspakets; nicht die lokal editierbare Applikation selbst.
Worker	Konfigurierte Zielsystem-Anbindung eines Worker Type mit Identität und Einstellungen; eine Worker Instance holt die Jobs ab.

Die Begriffe folgen dem Architekturglossar, ADR-0128, ADR-0203 und ADR-0499. Ein Produkt im Shop ist ebenfalls nicht mit einer Applikation gleichzusetzen: Ein Produkt kann einen Prozess starten oder Aktionen darüber anbieten; die Auslieferung seiner Katalogdefinition ist eine eigene, aber paketierbare Angelegenheit.

---

4. Architekturüberblick

Abbildung 1 · Einordnung von Applikationen oberhalb der Laufzeit-Engine



Umgesetzt: Die Applikation lebt als **Design-Time-Schicht** hinter HTTP-API, Console, CLI und MCP. Entwürfe, Releases, Bindungen und Importhistorie werden als eigenständige, dauerhaft gespeicherte Verwaltungsdaten geführt. Sie erscheinen **nicht** als Applikationsobjekte im Event-Log und werden **nicht** durch `applyToState` materialisiert. Das ist eine Architekturgrenze, kein blosses Darstellungsdetail. ADR-0034, ADR-0128, ADR-0019, Invarianten.

Beim Deployment wird das BPMN-Modell geprüft und in den unveränderlichen, integer-indexierten Ausführungsgraphen übersetzt. Eine Prozessinstanz arbeitet anschliessend mit dieser kompilierten Definition. Commands erzeugen Events; nach WAL-Schreibvorgang und `fsync` wird Zustand materialisiert und erst danach dürfen sichtbare Nebenwirkungen stattfinden. Der Prozessor wird je Partition von genau einem Writer betrieben. Das Applikationskonzept ändert diese Laufzeitmechanik nicht. Architektur, Invarianten.

Ein Worker führt externe Arbeit **ausserhalb des synchronen Engine-Hot-Paths** aus. Die Applikation kann Worker-Namen und benötigte Operationen deklarieren, sie liefert aber nicht automatisch die Zielsystem-Credentials, Worker-Prozesse oder Netzwerkfreigaben mit. Die Konfiguration einer Worker Instance ist vom Workflow-Modell und vom Worker Type zu unterscheiden. Weiterführend: Worker-Whitepaper, ADR-0208.

Einschätzung: Die klare Grenze zwischen Applikationsverwaltung und Event-Sourcing ist betrieblich wertvoll: Sie vereinfacht das Verpacken und Freigeben von Modellen, ohne den Replay-Vertrag der Engine durch nachträgliche Verwaltungsmetadaten aufzuweichen. Sie ersetzt jedoch kein gesondertes Recovery-Konzept für die Design-Time-Seitendaten.

---

5. Bestandteile einer Applikation

BPMN-Prozesse und DMN-Entscheidungen

Umgesetzt: BPMN kann als bearbeitbarer Entwurf gespeichert werden (ADR-0021). Das Publizieren einer Applikation validiert die zugeordneten Artefakte und legt ihre deployten Versionen in einem Release fest (ADR-0128). DMN hat eigene Entwürfe und dauerhaft versionierte Deployments: Speichern ist nicht gleich Veröffentlichen; eine DMN-Referenz bindet beim Prozessdeployment an eine geeignete Modellversion. Die Entscheide ADR-0319, ADR-0321, ADR-0322, ADR-0327 und ADR-0329 erläutern die Grenzen der jeweiligen Versionen.

Formulare und fachlicher Einstieg

Formulare beschreiben Eingaben und Aufgabenoberflächen und gehören zur fachlichen Anwendbarkeit eines Prozesses. Die Startansicht gruppiert veröffentlichte Einstiegsmöglichkeiten nach Applikation und hält persönliche Favoriten vor. **Umgesetzt** nach ADR-0466. Eine formal versionierte, veröffentlichte Prozessschnittstelle als eigenständiger Vertrag ist dagegen **geplant/offen**: ADR-0373 steht auf `Not started`.

Konstanten und Werte je Installation

Das Prozessmodell deklariert Konstanten, die beim Start einer Instanz gesetzt und danach vor gewöhnlichen Variablenänderungen geschützt werden. Lokale Werte können aus der Applikation oder der Installation stammen. Die Bindung erfolgt über explizite Namensräume wie `application.TENANT_ID` und `server.TENANT_ID`; vordefinierte Fakten wie `server.environment` können ebenso gelesen werden. **Umgesetzt** nach ADR-0453, ADR-0454 und ADR-0456.

Architektur, Informationsmodelle und Fähigkeiten

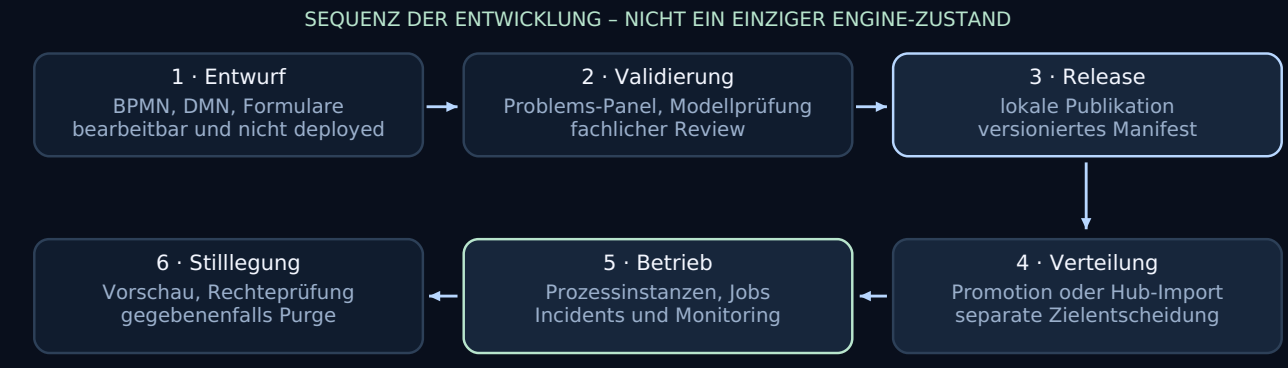
Umgesetzt mit Abgrenzung: Ein Release kann einen Architektur-Snapshot enthalten. Das Paketformat 4 enthält gemäss ADR-0528 zusätzlich Informationsmodelle und applikationsgebundene Realisierungen von Geschäftsfähigkeiten. Fähigkeiten und Wertströme bleiben dabei übergreifende Registereinträge der **Zielinstallation**; mitgelieferte Einträge sind Vorschläge und müssen bei Konflikten übernommen, zugeordnet oder ausgelassen werden. Das verhindert eine heimliche Überschreibung der lokalen Geschäftsfähigkeitslandkarte. Die vollständige mehrschichtige Darstellung und Veröffentlichung auf externen Plattformen bleibt gemäss ADR-0471 und ADR-0495 **teilweise umgesetzt**.

Worker-Abhängigkeiten und externe Systeme

BPMN-Service-Tasks brauchen zur Ausführung passende Worker und Zielberechtigungen. Ein Paket kann auf diese Abhängigkeiten hinweisen, darf aber nicht als umfassende, lauffähige Kopie der externen Infrastruktur verstanden werden. **Einschätzung:** Ein Release ist vollständig im Sinn seiner modellierten und mitgeführten Artefakte, nicht vollständig im Sinn aller operativen Abhängigkeiten. Bei Produktivfreigaben gehören Worker-Kompatibilität, Credential-Gültigkeit, Firewall-Regeln und Zielsystemrechte auf eine separate Checkliste. Siehe Worker-Whitepaper.

6. Lebenszyklus

Abbildung 2 · Lebenszyklus und bewusst getrennte Übergänge



Entwurf: Fachliche Modelle werden als nicht ausführungswirksame Entwürfe gespeichert. Unvollständige Modelle müssen noch nicht deploybar sein. **Validierung:** Vor dem Deployment werden Syntax, Modellsemantik und Bindungen geprüft. **Publikation:** Der lokale Publish-Vorgang erstellt die lauffähigen Versionen und das Applikations-Release. **Verteilung:** Promotion oder Paketimport übertragen ausgewählte Inhalte in andere Installationen. **Betrieb:** Die jeweilige Installation startet Instanzen, führt Jobs aus und meldet Incidents. **Stilllegung:** Unbenutzte Artefakte oder eine ganze Applikation werden gezielt gelöscht bzw. bereinigt.

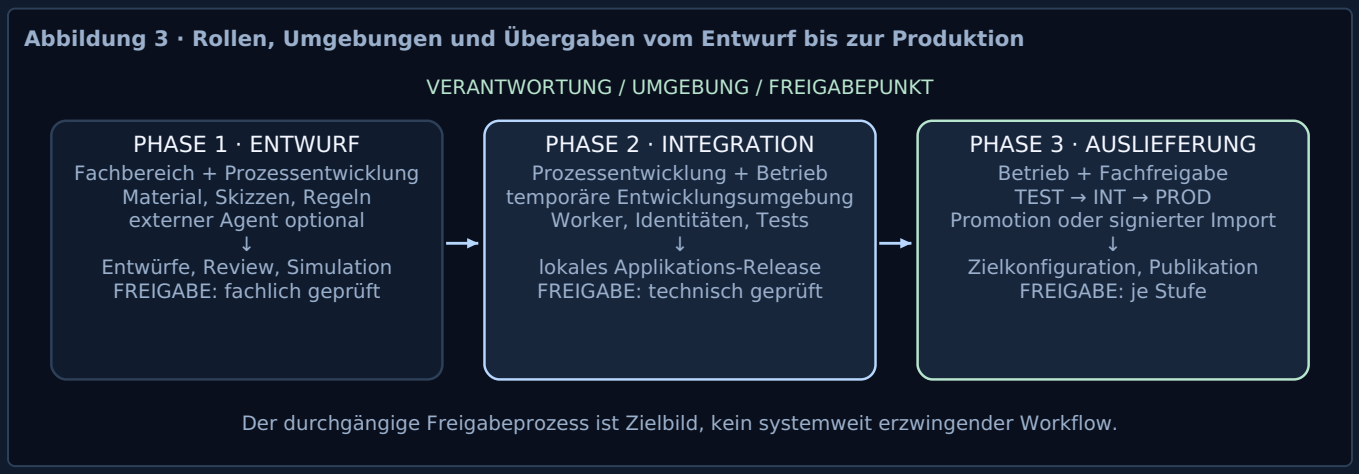
Das sind keine Zustände einer einzigen langlebigen Engine-Instanz; es sind Arbeitsschritte und Verwaltungszustände. Insbesondere bleibt ein schon gespeicherter Entwurf editierbar, während ein älteres Release weiterläuft. Ein neues Release ersetzt weder automatisch die Instanzhistorie noch beendet es sämtliche älteren Prozessinstanzen. ADR-0019, ADR-0128.

Umgesetzt: Der normale Applikations-Löschvorgang entfernt die Zuordnung und Verwaltungsdaten, nicht zwangsläufig die zugeordneten Artefakte. Ein ausdrücklich gewähltes `?purge=true` bietet eine Vorschau und einen weitergehenden Löschpfad. Er kann laufende Instanzen abbrechen und unterliegt zusätzlichen Eigentümer-, Schutz- und Konsistenzregeln. Der Vorgang ist somit potenziell destruktiv und niemals eine harmlose Aufräumaktion. ADR-0468, `api/apppurge.go`.

Einschätzung: Ein fachlicher Stilllegungsprozess sollte zuerst den Start neuer Instanzen und externe Aufrufe kontrollieren, dann aktive Fälle und Aufbewahrungspflichten klären und erst danach eine Löschung autorisieren. Eine Applikationsbereinigung ersetzt keine rechtliche Prüfung von Ereignisprotokollen und Geschäftsdaten.

7. Entwicklung einer Applikation

Dieses Kapitel trennt **drei Phasen**: fachlicher Entwurf, technische Integration sowie kontrollierte Auslieferung. Die unten beschriebenen Freigaben sind Arbeitsvorschläge, wo sie noch nicht durch eine Plattformfunktion technisch erzwungen werden. Lehrgang Prozessentwicklung.



Phase 1 – Fachlicher Entwurf, optional KI-gestützt

Rollen: Der Fachbereich trägt Prozessziel, fachliche Regeln und Abnahme. Die Prozessentwicklung strukturiert BPMN, DMN, Formulare und Informationsmodell. Der Betrieb unterstützt früh bei Identitäten, Umgebungen und Worker-Anforderungen. **Eingaben:** Prozessbeschreibungen, vorhandene Diagramme, Bilder, Papier-Skizzen, Formulare, Tabellen und bekannte Regeln. **Ergebnis:** nachvollziehbar gespeicherte **Entwürfe**, noch kein produktives Deployment. **Freigabepunkt:** fachliche Durchsicht des vorgesehenen Sollprozesses einschliesslich Rollen, Ausnahmen und Entscheidungen.

Umgesetzt: Externe MCP-fähige KI-Agenten können über die HTTP-orientierte MCP-Schicht Entwürfe lesen und schreiben. MCP arbeitet unter der Identität und mit den Rechten des authentisierten Aufrufers; je nach Client wird der Zugang über lokale Credentials oder OAuth ermöglicht. Das Speichern eines BPMN-Entwurfs ist ausdrücklich von einem Deployment getrennt. Gemeinsames Modellieren in Live-Sitzungen, DMN-Entwürfe, generierte Formularentwürfe und ein FEEL-Assistent besitzen eigene implementierte Bausteine. ADR-0016, ADR-0196, ADR-0200, ADR-0140, ADR-0260, ADR-0445.

Teilweise umgesetzt: Ein durchgängiger eingebauter, generativer Modeler-Copilot ist **nicht** als vollständig gelandete Funktion zu beschreiben: ADR-0032 steht auf `Proposed / Partial`. Die KI-Interpretation unstrukturierter Eingaben – etwa eines Fotos einer handgezeichneten Ablaufskizze – ist nach diesen Quellen eine Leistung des **externen KI-Agenten**, nicht eine zugesicherte automatische Funktion des atlas-Prozessors. Der externe Agent erzeugt oder ändert die editierbaren Modelle über dafür vorgesehene Schnittstellen. Aus diesem Grund gilt jede Generierung zunächst als Vorschlag.

Qualitätssicherung: Entwürfe werden im Modeler visuell geprüft; das Problems-Panel und die Deploy-Prüfung legen Modellierungsprobleme offen. Simulation beziehungsweise Play-Modus erlaubt, Pfade und Bedingungen vor echtem Betrieb zu untersuchen. Für DMN und FEEL ist die Prüfung gegen konkrete Eingaben erforderlich. Der Fachbereich muss Fälle, Ausnahmen, Zuständigkeiten und die Textwirkung in Formularen nachvollziehen. **Einschätzung:** Ein Agent kann Vorschläge beschleunigen, nicht die fachliche Verantwortung oder die spätere technische Abnahme übernehmen. Lehrgang, ADR-0032.

Risiken: Ein KI-Modell kann Schritte erfinden, Geschäftsregeln falsch interpretieren oder nicht vorhandene Mitarbeiterrollen zuordnen. Falls Ausgangstexte, Bilder oder Prozessbeispiele an einen externen KI-Anbieter gesendet werden, können vertrauliche Informationen die Organisationsgrenze verlassen. Authentisiertes MCP und die serverseitige Validierung begrenzen Zugriffs- und Formatfehler, verhindern aber keine inhaltliche Halluzination und ersetzen keine Datenklassifikation oder Genehmigung für einen Modellanbieter. **Einschätzung:** Für schutzbedürftige Unterlagen ist vor dem Upload ein genehmigtes Betriebs- und Datenverarbeitungsmodell erforderlich.

Phase 2 – Integration in der Entwicklungsumgebung

Rollen: Die Prozessentwicklung richtet Modelle, Worker-Bindungen und Testdaten ein. Der Betrieb stellt die temporäre Zielumgebung bereit und verwaltet Secrets, Netz, Identitäten sowie Zielsystemrechte. Der Fachbereich verantwortet fachliche Integrationstests. **Eingaben:** freigegebene Entwürfe, Zielsystem-Schnittstellen und Testkonten. **Ergebnis:** integrierte, lokal publizierte Applikation mit überprüften Abläufen und reproduzierbaren Abhängigkeiten. **Freigabepunkt:** dokumentierter Integrationsnachweis und Entscheidung, welches Release ausgeliefert wird.

Als konkret belegte Referenz dient `deploy/opentofu/m365-devenv/README.md`. OpenTofu erstellt darin eine temporäre Entwicklungsumgebung mit atlas-Server, Microsoft-365-Testumgebung beziehungsweise Testtenant, Identitäten und ausgewählten Workern. Skripte konfigurieren Secrets im Vault, registrieren konfigurierte Worker und spielen Beispielapplikationen ein. `tofu init`, `tofu apply` und `tofu destroy` bilden die wiederholbaren Infrastrukturphasen. Die Einspielskripte unter `deploy/opentofu/m365-devenv/scripts/` sind Teil des Referenzvorgehens.

Umgesetzt mit Grenzen: Die Beschreibung enthält einen dokumentierten Durchlauf vom **2. November 2026** in einem realen, geteilten Tenant, einschliesslich nachgeführter Korrekturen. Sie nennt gleichzeitig einen noch offenen zweiten vollständigen Neuaufbau, manuell notwendige Schritte sowie Funktionen, die nicht für einen geteilten Tenant geeignet sind. Eine vorhandene Integration wird dadurch nicht automatisch allgemein produktionsreif. Für Exchange, Teams Phone und SharePoint sind die Berechtigungsprofile nicht gleich: Einige Microsoft-API-Rechte gelten tenantweit, andere können enger beschränkt werden. Ohne geeignete Konfiguration sind sie zu weit für einen produktiv genutzten Tenant.

Die Entwicklungsumgebung ist **kein Produktions-Betriebsmuster**. Ihre eigenen Sicherheitsnotizen nennen unter anderem Secrets in OpenTofu-State, temporär verteilte Anmeldedaten und nach Konfiguration öffentlich erreichbare HTTPS-Endpunkte. Credentials sind kurzlebig; ihre genaue Laufzeit hängt von den Einstellungen ab. Ein geteiltes M365-Tenant-Setup reduziert mehrere Rechte, beseitigt aber nicht alle tenantweiten Rechte. **Einschätzung:** Das architektonische Muster – kurzlebige Umgebung, reproducible setup, minimal berechtigte Worker, realistische Integrationsfälle, vollständiger Rückbau – ist übertragbar. Die konkrete OpenTofu-Lösung belegt jedoch nicht automatisch Integrationen zu anderen Herstellern.

Praktische Prüfschritte: Vor dem ersten Publish muss klar sein, welche Service Tasks einen konfigurierten Worker brauchen, welche Konstanten die Applikation aus ihrer Installation liest, welche Nachrichten und Timer sie starten, welche DMN-Modelle sie bindet und wie Fehler als Incidents sichtbar werden. Danach müssen positive Fälle, Ablehnungen, Wiederholungen und Wiederanläufe getestet werden. Beim Wechsel von DEV nach TEST darf kein Entwicklungssecret mitwandern.

Phase 3 – Auslieferung nach Test, Integration und Produktion

Rollen: Die Prozessentwicklung liefert einen benannten, unveränderlichen Release-Kandidaten. Der Fachbereich bestätigt die fachliche Abnahme. Der Betrieb oder eine berechtigte Release-Verantwortung entscheidet über Einspielung, Konfiguration und Freigabe pro Zielinstallation. **Eingaben:** freigegebenes Release/Paket, Prüfnachweise, kompatible Worker und konkrete Zielumgebungen. **Ergebnis:** eine nachvollziehbare Zuordnung, welches Release in welcher Stufe publiziert wurde. **Freigabepunkt:** separat dokumentierte Entscheidung vor jedem Wechsel in die nächste Stufe.

Heute belegbarer Weg A – Promotion: Nach der lokalen Publikation wird ein vorhandenes Applikations-Release per Deployment-Ziel an einen anderen atlas-Server gesendet. Die Zielinstallation erhält die im Release eingefrorenen Modelle; der Zielserver verwendet einen dauerhaften, eng begrenzten Deploy-Token für die Publikationsoperation. Die Promotion meldet Erfolge und Fehler **je Ziel**; eine Promotion auf mehrere Ziele ist nicht als global atomare Aktion zugesichert. Die Release-Version reist mit und wird nicht auf dem Ziel neu nummeriert. **Umgesetzt** nach ADR-0129 und den Tests in `api/promote_test.go`.

Heute belegbarer Weg B – atlas hub: Auf der Herausgeberseite erzeugt die CLI mit `atlas hub keygen`, `atlas hub lock` und `atlas hub build` eine signierte Registry. Die Installation vertraut einer administrativ gepinnten öffentlichen Ed25519-Schlüsselmenge, lädt die überprüften Bytes und führt einen **Importplan** mit `dryRun`, Konfliktentscheidungen und gegebenenfalls nachfolgender Publikation aus. Der Import ist **nicht** dasselbe wie ein automatisches Deployment. Es existieren Console-, HTTP-, CLI- und MCP-Bezüge, jedoch nicht in jeder Kombination identische Automationsgrade. **Teilweise umgesetzt** im Gesamtsinn von ADR-0499; Details: Hub-Whitepaper.

Zielbild – noch nicht durchgehend implementiert: Ein Release wird signiert im atlas hub veröffentlicht, von TEST, INT und PROD jeweils über eigene kontrollierte Pull-Entscheidungen importiert und dort **separat** publiziert. Der Release-Digest bleibt gleich; jede Stufe protokolliert Genehmigung, Zielkonfiguration, Testergebnis und tatsächlich publizierte Prozessversionen. Eine weitere Freigabe folgt erst auf einen geprüften Stand. Diese durchgängige Kette ist eine **Einschätzung**, keine vorhandene automatische GitOps-/Release-Management-Pipeline.

Kritische Grenzen des Zielbildes: Nach dem dokumentierten Hub-Vertrag besitzt ein atlas-Server gegenwärtig **eine** fest konfigurierte Registry. Abrufe sind auf HTTPS, öffentlich auflösbare Hosts und Port 443 begrenzt; private Adressbereiche werden verweigert. Eine nur in internen Netzsegmenten erreichbare Registry lässt sich deshalb nicht ohne Architekturänderung als Standardquelle angeben. Öffentliche DNS-Hosts und ein zulässiger TLS-Endpunkt sind nicht automatisch die richtige Lösung für vertrauliche interne Artefakte. Dauerhafte Installations-Provenienz, Mehrquellenverwaltung und verwaltete Updates sind offen. Mehrere voneinander getrennte Ziele müssen daher ihren effektiven Stand zusätzlich mit externer Inventarisierung und Freigaben nachweisen. Hub-Whitepaper, ADR-0499, ADR-0506.

Der Lehrgang Prozessentwicklung nennt die Trennung der Umgebungen, Freigaben vor Promotion, Schreibschutz in Produktion, Verhältnis zu Git und Worker-Konfiguration je Stufe ausdrücklich als **offene Architekturentscheidung**. Dieses Whitepaper entscheidet sie nicht vorweg. **Einschätzung:** Bis zur verbindlichen Regelung sollte jede Organisation eine verantwortliche Freigabeinstanz, ein extern gesichertes Release-Inventar und eine dokumentierte Rückfallstrategie benennen.

---

8. Versionierung und Migration

Die Versionierung arbeitet auf mehreren Ebenen. Erstens besitzt die BPMN-Prozessdefinition ihre Version je Prozesskennung. Zweitens weist ein Applikations-Release eine eigene, fortlaufende Versionsnummer für den gemeinsam publizierten Artefaktstand aus. Drittens bezeichnet ein Blueprint-Release im atlas hub ein vom Herausgeber signiertes Distributionsobjekt mit eigenen Versions- und Digest-Identitäten. Diese Nummern sind nicht austauschbar. ADR-0019, ADR-0128, ADR-0499.

Umgesetzt: ADR-0128 beschreibt Applikations-Releases als lokale Manifest-Einträge mit Release-Version, Mitgliedsartefakten und zugehörigen Prozessversionen, Zeit und verantwortlicher Person. Ein Release entsteht beim Publizieren. Das Vorhandensein einer neueren Applikationsversion bedeutet nicht, dass eine bereits laufende Instanz auf eine neue Prozessdefinition umgeschaltet wird. Die Engine benötigt die Zuordnung einer konkreten Prozessinstanz zu ihrer Definition, damit Event-Replay und die Ausführung deterministisch bleiben.

Umgesetzt: Bei der Promotion über ADR-0129 wird die vom Ausgangsserver vergebene Release-Version mitgeschickt. Der Zielserver verweigert eine gleich nummerierte, aber inhaltlich andere Version, anstatt stillschweigend umzunummerieren. Ein Ziel, auf dem gleichzeitig lokal veröffentlicht wird, braucht deshalb organisatorische Regeln gegen konkurrierende Release-Quellen. Die technische Promotion ist kein Merge-Verfahren für zwei unabhängige Autorenstände.

Umgesetzt: Der Paketimport kann wiederholt werden, ohne unveränderte Artefakte neu zu schreiben. Seit ADR-0467 merkt sich die Zielinstallation für importierte Artefakte eine **Baseline**. Dadurch unterscheidet der Plan, ob nur die neue Quelle fortgeschritten ist (`behind`), nur lokal editiert wurde (`ahead`) oder beide Seiten Änderungen enthalten (`changed`). Diese Dreierbeziehung schützt lokale Änderungen besser als ein pauschales Überschreiben. Ein Importstatus beschreibt allerdings Artefaktunterschiede; er beweist noch keine erfolgreiche fachliche Migration des Prozesszustands.

Geplant/offen: Ein allgemeiner Applikations-Migrationsvertrag für laufende Prozessinstanzen folgt aus den genannten Paket- und Hub-Funktionen **nicht**. Ein Hub-Update migriert laut Hub-Whitepaper keine laufenden Instanzen. Die in ADR-0453 beschriebenen gebundenen Konstanten einer Instanz werden auch bei einer Prozessmigration nicht neu vom Server gelesen. Betriebsverantwortliche müssen deshalb selbst unterscheiden zwischen: neue Starts auf neuer Definition, Fertigstellen alter Instanzen und explizit geregelter Migration.

Einschätzung: Für lang laufende Vorgänge sollte eine Freigabeakte die Kompatibilität geänderter Formulare, Nachrichten, DMN-Ergebnisse, Worker-Aufrufe und Datenstrukturen dokumentieren. Eine technische Erfolgsantwort des Importers ist nicht gleichzusetzen mit fachlicher Rückwärtskompatibilität. Besonders bei wiederholten Service Tasks und bereits vergebenen User Tasks sind unerwartete Seitenwirkungen getrennt zu bewerten.

---

9. Konfiguration und Konstanten

Eine Applikation soll zwischen Entwicklung, Test, Integration und Produktion transportierbar sein, ohne für jede Stufe fachlich neue BPMN-Dateien zu benötigen. Sie darf dabei nicht die Sicherheitsidentität und die Zielsysteme einer Testinstallation als produktive Wahrheit mitliefern. Die Trennung von **deklarierter Abhängigkeit** und **lokal gebundenem Wert** löst einen Teil dieses Problems. ADR-0453, ADR-0456.

Konfiguration	Entwicklungsumgebung	Test/Integration	Produktion	Im Paket?
BPMN-/DMN-Modell, Formulare	Entwurf und Probeläufe	freigegebene Release-Version	freigegebene Release-Version	Ja, soweit zugeordnet
`application.TENANT_ID`	Testtenant	Stufentenant	Produktivtenant	Als Abhängigkeit; Wert nur kontrolliert als Vorschlag
`server.environment`	z. B. DEV	TEST / INT	PROD	Nein: Fakt der Zielinstallation
Worker-Name und benötigte Operation	Test-Worker	stufenspezifische Worker	Produktiv-Worker	Referenz, nicht laufende Worker-Identität
Zugangsdaten, Zertifikate, API-Secrets	kurzlebige Testwerte	eigene verwaltete Secrets	eigene Secrets mit Least Privilege	Nicht als frei übertragbare Laufzeit-Credentials
Identitätszuordnungen und Freigebende	Testgruppen	stufengerechte Zuordnung	produktive Gruppen	Fragen und Zuordnungsentscheidungen
Katalog und Produktdefinitionen	Testangebote	Abnahmeangebote	freigegebene Angebote	Optional gemeinsam mit der Applikation

Umgesetzt: `application.X` und `server.X` sind deklarierte, unterscheidbare Namensräume. Zur Instanzstartzeit werden die gebundenen Werte in den Instanzzustand aufgenommen und dürfen nicht einfach durch spätere Variablenupdates umgeschrieben werden. Die Architektur schützt dadurch den deterministischen Replay-Zustand: Die Quelle darf sich nach dem Start ändern, aber die Instanz behält den für sie erfassten Wert. Vordefinierte Fakten der Installation (`server.id`, `server.name`, `server.environment`) und der Applikation (`application.id`, `.key`, `.name`) können im Modell deklariert werden. ADR-0454, ADR-0456.

Nicht jeder fehlende Wert wird auf jedem Startpfad gleich behandelt. Der dokumentierte Vertrag unterscheidet einen manuell zurückweisbaren Start von Timer-, Nachrichten- oder Signalstarts, bei denen ein Trigger nicht einfach verloren gehen darf. Warnungen im Deploy- oder Konfigurationskontext sind deshalb ein wichtiges, aber nicht vollständiges Schutzinstrument. Ein Zertifikat, das sich während einer laufenden Instanz drehen kann, gehört nach ADR-0453 zur Auflösung beim Worker-Aufruf, nicht in eine unveränderlich beim Prozessstart eingefrorene Konstante.

Umgesetzt: Paketkonstanten werden in `konstanten.json` als Vorschläge transportiert. Ein lokal anderer Wert wird nicht unbesehen überschrieben. Der Import verwendet Konfliktentscheidungen; das Setzen von Konstanten braucht administrative Berechtigung. Ist die Berechtigung nicht vorhanden, benennt der Import die noch offene Einrichtung, statt diese stillschweigend als erledigt zu melden. `api/constantsimport.go` und `api/constantsimport_test.go` prüfen diesen Pfad. ADR-0467.

Einschätzung: Ein Umgebungsinventar sollte pro Release mindestens benötigte Namespace-Namen, Worker, Secret-Referenzen, Zielsysteme, verantwortliche Teams und technische Verifikationsfälle führen. Diese zusätzlichen Angaben sind Betriebsartefakte; sie dürfen nicht als automatisch durch alle heutigen atlas-Paketformate abgegolten gelten.

---

10. Verteilung

Abbildung 4 · Zwei getrennte Verteilungswege mit unterschiedlichen Vertrauensrichtungen



Hub-Paketimport und direkte Promotion sind alternative, nicht identische Vertrauenswege.

Paketexport und Paketimport

Umgesetzt: `atlas export --application KEY [--catalog ID]... DIR` erzeugt ein Paketverzeichnis. Dazu gehören je nach Inhalt `atlas.json` und `processes/`, `forms/`, `decisions/`, ausserdem `konstanten.json`, `katalog.json`, `fragen.json` und `antworten.json`. Der Quellbaum bildet bearbeitbare Artefakte ab, während die separat mitgeführten Konstanten als installationsbezogene Vorschläge behandelt werden. Die Fragen abstrahieren Zielgruppen und Genehmigende, deren lokale IDs auf einer anderen Installation nicht dieselbe Bedeutung haben. ADR-0467.

Umgesetzt: `atlas import DIR` kann den Importplan anzeigen und in interaktiven oder nichtinteraktiven Situationen gezielte Entscheidungen treffen. `--dry-run` schreibt nichts. Ein ungeklärter Unterschied führt zu `409`, nicht zu einem versteckten Overwrite. `overwrite`, `keep` und bei Artefakten anderer Applikationen das explizite `adopt` sind getrennte Entscheidungen. Die jüngste ADR-0467-Erweiterung schützt zudem gemeinsam genutzte DMN-Modelle: Sind Entscheidungen ausserhalb der Applikation betroffen, reicht ein globales `overwrite=*` nicht. Es braucht die **namentliche** Entscheidung und Bearbeitungsrechte auf betroffenen Applikationen. Die Tests der Paket- und Import-Endpunkte sind dafür relevante Umsetzungsevidenz.

Umgesetzt: Mit ADR-0528 wurde das Quellbaumformat um Informationsmodelle und Architekturangaben erweitert. Nicht vorhandene Fähigkeiten im serverweiten Register werden beim Import als Lücke behandelt. Die importing verantwortliche Person kann einen Vorschlag ausdrücklich übernehmen, einem existierenden Schlüssel zuordnen oder auslassen. Eine ältere Paketbeschreibung darf solche neuen Felder nicht unbemerkt ignorieren. Dies ist eine wichtige Differenz zwischen dem aktuellen Code- und ADR-Stand und älteren Beschreibungen der Quellbaumstruktur.

atlas hub – Pull und signierte Releases

Umgesetzt: Die signierte Blueprint Registry v1 verwendet einen Registry-Index, Ed25519-Signaturen, SHA-256-Prüfsummen der Archive und eine vom Betreiber konfigurierte Vertrauensbasis. Die CLI-Werkzeuge `atlas hub keygen`, `lock` und `build` unterstützen die Herausgeberseite. Die Zielinstallation wählt in der Console einen Eintrag, fordert eine Vorschau an und prüft die Signatur, die erwarteten Hashes sowie Archivstruktur und Grenzen. Der Paketbezug kann an einen Digest gebunden werden, sodass ein zwischen Auswahl und Bezug geänderter Stand nicht unbemerkt installiert wird. Hub-Whitepaper, ADR-0499.

Teilweise umgesetzt: Der signierte Transport bildet noch **keine** vollständige System-of-Record-Sicht, welche Quelle, Signaturidentität, Blueprint-Version, lokal publizierte Applikationsversion und Prozessversionen jeder Umgebung dauerhaft verbindet. Mehrere Registries, vollständige Versionshistorie, automatische Updates und installationsübergreifende Release-Governance sind nicht als fertig anzunehmen. ADR-0499, ADR-0506.

Direktes Promotion-Ziel – Push und Deploy-Token

Umgesetzt: ADR-0129 sieht einen ausgehenden, zielgerichteten Push des bereits publizierten Releases vor. Die Zielinstallation stellt einen eingeschränkten Deploy-Token bereit, den der sendende Server in seinem Vault referenziert. Die Zieldefinition trennt bei Bedarf Publikations-Credential und ein optionales Leserecht für Zielstatus und Landschaftsinformation. Der sendende Server berichtet pro Ziel über den Ausgang. Ein Fehler auf einem Ziel verhindert nicht zwingend die erfolgreiche Promotion auf andere Ziele; es gibt keinen verteilten Alles-oder-nichts-Commit.

Kriterium	atlas hub (Pull)	Deployment-Ziel (Push)
Auslöser	Zielinstallation wählt signiertes Paket	Ausgangsinstallation sendet eingefrorenes Release
Vertrauen	Gepinnter Herausgeberschlüssel und Digest; Ziel prüft Bytes	Ziel vergibt eng begrenzten Deploy-Token an Sender
Netzrichtung	Ziel muss Registry erreichen können	Sender muss Zielserver erreichen können
Import-/Freigabeschritt	Vorschau, Konfliktentscheidung, separate Publikation	Remote-Import/Deployment des Releases als Promotion
Geeignet, wenn …	unabhängige Empfänger Pull selbst kontrollieren sollen	Server gekoppelt sind und ein freigegebener Sender Releases verteilt
Grenze	eine Quelle, öffentliches DNS/HTTPS:443, offene Provenienz/Update-Funktionen	Sender benötigt erreichbaren Zielserver; Zielberechtigungen und Mehrziel-Fehler separat führen
ADR-Umsetzung	ADR-0499: **Partial**	ADR-0129: **Landed**

Einschätzung: Der Hub-Weg begrenzt die Annahme unbekannter Publisher durch Signaturprüfung und einen bewusst festen Importvertrag. Der Promotion-Weg ist überzeugend, wenn ein zentraler Release-Prozess die Hoheit über Zieldeployments besitzen soll und die Quellinstallation verlässlich erreichbar ist. Bei strikt isolierten Netzzonen darf die Wahl nicht allein nach Bedienkomfort fallen: Ein Pull-Client, der nur öffentlich adressierte Registries akzeptiert, passt nicht ohne Änderung zu rein privaten Netzen; ein Push-Server scheitert, wenn das Ziel keinerlei eingehenden Zugang erlaubt. Das ist eine Architekturentscheidung mit Netz- und Sicherheitsverantwortung.

11. Sicherheit und Berechtigungen

Applikationsbearbeitung, Publikation, Import, Betrieb und das Löschen sind unterschiedliche Handlungen. Sie müssen mit den jeweiligen Rechten der identifizierten Person oder des Dienstes erfolgen. **Umgesetzt:** Authentisiertes MCP repräsentiert den Aufrufer und benutzt die öffentlichen HTTP-Schnittstellen, statt direkte Manipulationen am Engine-Zustand vorzunehmen. Ein MCP-fähiger KI-Agent erhält damit keine implizite Sonderrolle. Ein Benutzer ohne Bearbeitungsrecht auf eine Applikation darf auch über einen Agenten nicht deren Inhalte überschreiben. ADR-0196, ADR-0200.

Umgesetzt: Beim Paketimport sind Fremdeigentum und geteilte Entscheidungsmodelle besonders geschützt. Ein `adopt` verlangt Bearbeitungsrecht auf der Quellapplikation. Das namentliche Überschreiben eines DMN-Modells mit externen Referenzen verlangt Bearbeitungsrechte auf den relevanten Anwendungen. Eine als geschützt gekennzeichnete Applikation gibt solche Artefakte nicht frei. Der Importer meldet nicht jeder unberechtigten Person fremde Applikationsdetails. ADR-0467.

Umgesetzt: Beim direkten Promotion-Weg ist das Deploy-Token kein Administratorpasswort. Das schützt gegen unnötige Delegation von Vollzugriff. Für lesende Peer-Operationen kann ein anderes Credential nötig sein. Im Hub-System sind die veröffentlichten Bytes nur bei intakter Schlüsselverwaltung, korrekter Betreiberkonfiguration und wirksamen URL-/Archivprüfungen vertrauenswürdig. Signaturen schützen Integrität und Schlüsselbindung, nicht die fachliche Richtigkeit oder die Bösartigkeit zulässiger Modellinhalte. ADR-0129, Hub-Whitepaper.

Einschätzung – Bedrohungen und Gegenmassnahmen: Vor allem fünf Grenzen sind organisatorisch zu prüfen. Erstens können Artefakte aus nicht genehmigten KI-Datenquellen vertrauliche Informationen enthalten. Zweitens kann ein Prozess legitime Worker mit zu weitreichenden Credentials missbrauchen. Drittens kann ein korrekt signiertes Paket unerwünschte Geschäftslogik mitbringen. Viertens können Fehlkonfigurationen in einer Produktivstufe auf echte Empfänger, Berechtigungen oder Daten zielen. Fünftens kann eine destruktive Purge-Aktion historische Arbeitsgegenstände beseitigen. Erforderlich sind ein genehmigter KI-Kanal, Quell- und Code-Review, Least-Privilege-Worker, isolierte Testsysteme sowie getrennte Berechtigungen für Publikation und Betrieb. Nicht jede dieser Kontrollen wird von atlas selbst technisch erzwungen.

Geplant/offen: Für ein produktweites, auditierbares Freigabeverfahren über TEST, INT und PROD existiert nach dem Lehrgang noch keine vollständig beschlossene Referenzarchitektur. Ebenso ist die implementierte Hub-Signaturprüfung keine Zertifizierung der Herkunftsorganisation, kein Software-Supply-Chain-Audit und keine Aussage über Konformität mit einer behördlichen Sicherheitsnorm.

12. Betrieb

Die Console bündelt Entwurf, Applikationsansicht, Releases, Konfiguration und Diagnose. Die HTTP-API ist die Grundlage für CLI- und MCP-Aktionen; ein Automationsskript soll dieselben öffentlich kontrollierten Operationen verwenden, nicht interne Stores direkt verändern. Der Betrieb benötigt zusätzlich Sicht auf Prozessdefinitionen, laufende Instanzen, User Tasks, Worker-Jobs und Incidents. Architektur, ADR-0016.

Umgesetzt: Die Startansicht gruppiert Prozesse nach Applikation. Releases erhalten Architekturinformationen; die Oberfläche kann gespeicherte Perspektiven zur Release-Struktur anzeigen. Promotion-Handler bieten Zielstatus und Fehlerrückmeldungen. Beim Löschen stehen Vorschau und expliziter Bereinigungsmodus zur Verfügung. Die Engine selbst persistiert Zustandsänderungen vor Rückmeldungen und externen Nebenwirkungen, wie es die Invarianten verlangen. ADR-0466, ADR-0471, ADR-0468.

Teilweise umgesetzt: Ein Architektur-Snapshot ist eine Aufnahme beim Release und keine permanent aktualisierte Topologie oder vollständige Risikoanalyse. Einige Darstellungs- und Exportfunktionen sind nach ADR-0471 und ADR-0495 weiterhin offen. Ein ausgelesener Zielstatus kann zudem nur belegen, was das jeweilige Ziel zum Anfragezeitpunkt zurückmeldet; er ersetzt keinen abgestimmten, langfristigen Release- und Abnahme-Nachweis.

Einschätzung – minimaler Betriebsdatensatz: Sinnvoll sind Applikationsschlüssel, lokale Release-Version, Blueprint-Digest (falls Hub), importierte Artefaktbaselines, Prozessversionszuordnung, Zielinstallation, tatsächliches Publikationsdatum, fachliche Freigabe, technische Freigabe, Worker-Konfiguration, Abhängigkeiten, offener Incident-Bestand und Rückfallbeschreibung. Der Datensatz kann vorläufig in einem externen Betriebsjournal geführt werden. Er ist ausdrücklich **kein** als vollständig implementiertes Produktfeature behauptetes Dashboard.

Bei Störungen ist zwischen Fehlern der **Verteilung** (Registry-Ausfall, Digest-Wechsel, Token verweigert, Ziel unerreichbar), Fehlern der **Einrichtung** (fehlender Worker, fehlende Konstante, nicht vorhandene Zielgruppen) und Fehlern **in der Laufzeit** (Jobs, Incidents, externe APIs) zu unterscheiden. Ein fehlgeschlagener Import ist nicht dieselbe Klasse von Problem wie ein Incident einer bereits laufenden Instanz. Wiederholungen und Fehlerbehandlung folgen daher unterschiedlichen Zuständigkeiten.

---

13. Durchgehendes Fallbeispiel

Das öffentlich dokumentierte Beispiel `examples/m365-dienste` demonstriert eine Applikation für Microsoft-365-Dienstleistungen. Es verbindet Prozessabläufe, Formulare, Entscheidungsmodelle und Katalogangebote mit Workern für externe Zielsysteme. Die separat dokumentierte `m365-devenv` macht diese Beispielwelt mit einer wiederholt aufbaubaren Entwicklungsumgebung konkret. Die Beispiele sind **Anwendungs- und Testmaterial**, keine Freigabe für den ungeprüften Einsatz in einer beliebigen produktiven M365-Organisation.

Schritt 1 – Fachlicher Entwurf: Der Fachbereich beschreibt etwa die Anforderung «Microsoft-365-Lizenz zuweisen», die nötigen Eingaben, Zuständigkeiten und Fälle der Ablehnung. Die Prozessentwicklung modelliert einen BPMN-Ablauf, ein Bestellformular, gegebenenfalls eine DMN-Entscheidung und die benötigte Worker-Operation. Die M365-Beispiele enthalten bereits verschiedene Aktionen zu Lizenzen, Postfächern, Teams und anderen Diensten. **Einschätzung:** Wo ein externer Agent bei der Erstmodellierung hilft, sind seine Entwürfe separat zu prüfen, insbesondere Berechtigungsannahmen und sensible Personendaten.

Schritt 2 – Entwicklungsinstallation: OpenTofu baut die vorgesehene Umgebung auf; ein Einspielskript aus `deploy/opentofu/m365-devenv/scripts/` bereitet das Beispiel auf ihre Tenant-Werte vor und installiert es. Ein Bootstrap-Skript hinterlegt Worker-Credentials im Vault und registriert die Worker. Die Konfiguration verwendet Test-Identitäten, stufenspezifische Konstanten und die zulässigen API-Rechte. **Umgesetzt:** Der dokumentierte Durchlauf mit echtem Tenant belegt wichtige Teile dieses Szenarios. **Nicht belegt:** dass alle M365-Berechtigungen in jedem geteilten Tenant hinreichend eng begrenzbar sind.

Schritt 3 – Integrationstest und lokales Release: Über Startansichten, Katalog oder API wird der Prozess ausgelöst. Die Person prüft Eingabefelder, Zuständigkeiten, DMN-Ergebnisse und Rückmeldungen des Zielsystems. Fehlkonfigurationen können zu zurückgewiesenen Starts oder Incidents führen. Nach Tests wird die gesamte Applikation publiziert. Ihr Release hält fest, welche Prozessversionen gemeinsam freigegeben wurden. Die Laufzeitdaten der Instanzen bleiben davon getrennt.

Schritt 4 – Transport: Das Paket kann auf dem Ausgangssystem mit dem Exportpfad vorbereitet und auf dem Ziel über den Importplan eingespielt werden. Konkrete Tenant-IDs und Gruppen werden auf dem Ziel zugeordnet, nicht ungeprüft übernommen. Für einen verbundenen Zielserver ist alternativ die direkte Promotion des eingefrorenen Releases möglich. Für unabhängige Empfänger ist ein signiertes atlas-hub-Release denkbar, sofern die heute gültigen Registry- und Netzvorgaben eingehalten werden.

Schritt 5 – Freigaben je Stufe: In TEST und INT werden eigene Worker, Berechtigungen und Konstanten eingerichtet. Die Verantwortlichen dokumentieren Ergebnisse und bestätigen das fortgeschriebene Release. Für PROD wird der Entscheid zur Publikation separat getroffen. **Geplant/offen:** Diese gesamte Freigabekette wird nicht als systemseitig durchgesetzter Standard behauptet. Nachweise über den produktiven M365-Tenant, Administrationsgrenzen und minimale Worker-Rechte sind ausserhalb des Paketimports zu erbringen.

Schritt 6 – Betrieb und Änderung: Neue Anforderungen führen zu einem neuen Entwurf und anschliessend zu einem weiteren Release. Der Importplan erkennt örtliche Abweichungen. Laufende alte Instanzen werden nicht durch den blossen Hub-Import migriert. Worker-Rechte und Zertifikate können sich unabhängig von den modellierten Konstanten ändern. Bei einer Stilllegung ist eine Vorschau des Bereinigungsumfangs erforderlich, bevor über das Abbrechen laufender Instanzen entschieden wird.

Als kleineres Alternativbeispiel dokumentiert `examples/lizenzmanagement` die Verwaltung von Lizenzverträgen und Abonnements mit langen laufenden Instanzen. Das unterstreicht, warum eine neue Applikationsversion nicht gleichbedeutend mit einer sofortigen Umstellung aller Geschäftsobjekte ist.

---

14. Offene Punkte, Grenzen und Ausblick

Gegenstand	Quellenlage	Einordnung
Git-Repository als bidirektionale Applikationsquelle	Source-Tree-Export/Import vorhanden; echte Bindung, Sync, Commit und Push fehlen. ADR-0134, Code `api/appsource.go`.	**Teilweise umgesetzt**
Vollständiger Modeler-Copilot	Externe MCP-Agenten und spezialisierte Assistenten vorhanden, Gesamt-ADR-0032 `Proposed / Partial`.	**Teilweise umgesetzt**
Formale Prozessschnittstelle	ADR-0373 `Accepted / Not started`.	**Geplant/offen**
Hub-Provenienz und Mehrquellenfähigkeit	ADR-0499 `Partial`, Ein-Registry- und Netzschutz im Hub-Whitepaper; kein vollständiger Stand-Nachweis über alle Stufen.	**Teilweise umgesetzt**
Verwaltete Updates und Katalog-only-Offers	ADR-0506 `Proposed / Not started`; keine stillschweigende automatische Publikation.	**Geplant/offen**
Vollständige Architektursichten und Confluence-Publikation	ADR-0471 und ADR-0495 beide `Partial`; Snapshot-Funktionen vorhanden.	**Teilweise umgesetzt**
Architekturinhalt im Paket	ADR-0528 `Landed` mit Quellbaumformat 4 und bewussten Importentscheidungen für Fähigkeiten.	**Umgesetzt**
M365-Testumgebung	Ein realer Durchlauf dokumentiert; zweiter vollständiger Aufbau und einzelne Integrationspunkte offen.	**Umgesetzt mit begrenzter Evidenz**
Verbindliche DEV→TEST→INT→PROD-Governance	Als offene Entscheidung im Lehrgang dokumentiert.	**Geplant/offen**
Gemeinsame Capability Map über Servergrenzen	ADR-0528 nennt eine offene Frage zwischen servereigenen Karten und zentral geteilter Landkarte.	**Geplant/offen**

Ein **Widerspruch in der Dokumentation** ist ausdrücklich festzuhalten: Der Abschnitt «Was nicht bekannt ist» im Lehrgang beschreibt ADR-0134 noch als «Landed», obwohl der aktuelle ADR-Index und der ADR selbst `Partial` festhalten. Für dieses Whitepaper gilt der verifizierte `Partial`-Status; die ältere Lehrgangsformulierung ist zu berichtigen, ohne das nicht implementierte Git-Binding als fertig zu behandeln.

Die Quellen für die Hub-Verteilung, insbesondere ein älterer «As built»-Abschnitt von ADR-0499 und das aktuelle Hub-Whitepaper, enthalten zeitlich unterschiedliche Momentaufnahmen. Der ADR-Index bleibt `Partial`, unabhängig davon, dass spätere Einzel-PRs einen früher noch offenen Teil nachgerüstet haben. Für einen tatsächlichen Betrieb ist jeweils zu prüfen, welche Version des Binärpakets installiert ist und ob die zugehörigen Funktionen konkret verfügbar sind. **Einschätzung:** Ein automatischer Release-Standbericht sollte deshalb nie nur einen ADR-Status abfragen, sondern Repository-Commit, Binärversion, anwendbare Migrationen, Zielkonfiguration und getesteten Importpfad verbinden.

Offen zu klären: Welche Instanz hält den verbindlichen Freigabeentscheid? Wie wird die Produktionsschreibsperre technisch und organisatorisch geregelt? Welche Menge an Worker- und Credential-Metadaten ist Teil eines releasefähigen Inventars? Wie wird bei isolierten Netzen eine private, signierte Registry angebunden, ohne die bestehenden SSRF-Schutzvorgaben zu schwächen? Wie werden Genehmigungen und Digest/Version über die vier Stufen revisionsfest verbunden? Welche Instanz ist für die organisationsweite Capability Map massgeblich? Diese Fragen sind vor einer verbindlichen Produktivstandardisierung in separaten Architektur- und Betriebsvorgaben zu beantworten.

---

15. Glossar und ADR-Verzeichnis

15.1 Leseschlüssel und Quellen

Umgesetzt – konkret belegter Implementierungsteil; **Teilweise umgesetzt** – nur ein Teil des beschlossenen Umfangs ist vorhanden; **Geplant/offen** – weder als fertige Funktion noch als verbindlich beschlossener Prozess nachgewiesen; **Einschätzung** – redaktionelle Bewertung. Für Begriffe siehe zusätzlich Architekturglossar. Zum Implementierungsstatus gilt der ADR-Index, ergänzt um die jeweiligen «As built»-Abschnitte und den Statusabgleich vom 11. Oktober 2026.

15.2 Referenzierte Architekturentscheide

ADR	Thema	Status	Implementation
0016	MCP über HTTP-API	Accepted	Landed
0019	Dauerhafte Deployments	Accepted	Landed
0021	BPMN-Entwürfe	Accepted	Landed
0032	Modeler-Copilot	Proposed	Partial
0034	Projekte und Artefakte	Accepted	Landed
0128	Prozessapplikationen	Accepted	Partial
0129	Entfernte Deployment-Ziele	Accepted	Landed
0134	Git-gestützte Applikationen	Accepted	Partial
0140	Gemeinsames Modellieren	Accepted	Landed
0196	Authentisiertes MCP	Accepted	Landed
0200	MCP-OAuth-Resource-Server	Accepted	Landed
0208	Worker-Type-Pakete	Accepted	Partial
0260	KI-Formularerstellung	Accepted	Landed
0319	Versionierte DMN-Deployments	Accepted	Landed

ADR	Thema	Status	Implementation
0321	DMN-Entwürfe	Accepted	Landed
0322	Einzelnes DMN-Deployment	Accepted	Landed
0327	Neueste DMN-Bindung	Accepted	Landed
0329	Löschschutz DMN-Deployment	Accepted	Landed
0373	Publizierte Prozessschnittstelle	Accepted	Not started
0437	Import via CLI	Accepted	Landed
0445	FEEL-Assistent	Accepted	Landed
0453	Applikationswerte je Installation	Accepted (amended)	Landed
0454	Prozesskonstanten	Accepted (amended)	Landed
0456	Konstanten-Namensräume	Accepted	Landed
0466	Startansicht nach Applikation	Accepted	Landed
0467	Applikationspaket / Konflikte	Accepted (amended)	Landed
0468	Bereinigendes Löschen	Accepted	Landed
0471	Release-Architektur-Snapshot	Accepted (amended)	Partial
0495	Architektur-Schichten	Accepted (amended)	Partial
0499	atlas hub Registry	Accepted	Partial
0506	Katalog-Bundles und Updates	Proposed	Not started
0520	Bestellung im Namen der Applikation	Accepted	Landed
0528	Architekturinhalt im Paket	Accepted	Landed

15.3 Weiterführende Unterlagen

Architekturübersicht, Architekturinvarianten, Prozessapplikationsplan, M365-Entwicklungsumgebung, M365-Beispielapplikation, Lizenzmanagement-Beispiel, Lehrgang Prozessentwicklung, Hub-Whitepaper, Worker-Whitepaper, Brand-Regeln.

Faktenbasis: Repository-Hauptzweig, Stand 6. November 2026. Dieses Whitepaper erklärt Status und Architektur, ist aber weder ein selbständiger Security-Audit noch ein Abnahmenachweis für produktive Nutzung.

15.4 Prüfprotokoll für eine konkrete Einführung (Einschätzung)

Die folgenden Prüffragen bilden eine **redaktionelle Empfehlung** für einen Implementierungs- und Freigabenachweis. Sie sind kein bereits eingebauter Freigabe-Assistent. Die Fragen müssen für jede Organisation mit ihren internen Sicherheits-, Datenschutz- und Betriebsregeln abgeglichen werden.

1. **Entwurfsherkunft.** Welche Person hat BPMN, DMN, Formulare und importierte Quellen fachlich freigegeben? Wurde bei KI-Unterstützung festgehalten, welcher Anbieter Inhalte verarbeitet hat, welche Schutzklasse die Unterlagen besitzen und ob Änderungen am Entwurf tatsächlich geprüft wurden?
2. **Release-Identität.** Welche Applikationskennung, welche lokale Release-Version, welche enthaltenen Prozess- und DMN-Versionen sowie gegebenenfalls welcher Hub-Digest bilden den freizugebenden Stand? Ist ein Vergleich mit einem früheren Release möglich, ohne einen alten Entwurf zu überschreiben?
3. **Umgebungsabhängigkeiten.** Welche `application.*`- und `server.*`-Konstanten liest das Modell? Welche Werte sind auf TEST, INT und PROD bewusst unterschiedlich? Ist ein fehlender Wert ein technischer Fehler oder darf die definierte Fallback-Semantik verwendet werden?

4. **Worker und Identitäten.** Sind die erforderlichen Worker Types und konfigurierten Worker vorhanden, ihre Operationen kompatibel und die verwendeten Credentials gültig? Ist der effektive API-Berechtigungsumfang pro Zielsystem dokumentiert, insbesondere bei tenantweiten Microsoft-Graph-Berechtigungen?
5. **Integrationsnachweis.** Wurden fachliche Positiv- und Negativfälle, Benachrichtigungen, Fristen, DMN-Entscheidungen, Wiederholungen, Incident-Bearbeitung und mindestens ein geplanter Wiederanlauf durchgespielt? Ist klar, welche Prüfungen ohne echte Zielsysteme lediglich simuliert wurden?
6. **Transport- und Vertrauensweg.** Erfolgt die Auslieferung über einen geprüften, begrenzt berechtigten Push oder über den signierten Pull? Bei atlas hub: Stimmen vertrauenswürdiger Schlüssel, Digest und Registry-Quelle? Bei Promotion: Ist der richtige Zielserver adressiert, und ist der Deploy-Token tatsächlich nur für den beabsichtigten Zweck gültig?
7. **Importkonflikte.** Welche vorhandenen Artefakte bleiben unverändert, welche werden überschrieben und welche müssen aus anderen Applikationen explizit übernommen werden? Betreffen DMN-Modelländerungen Entscheidungen ausserhalb der Applikation? Wer besitzt die dazu erforderlichen Rechte und die fachliche Verantwortung?
8. **Betriebsübergabe.** Ist die publizierte Version in einem nachvollziehbaren Inventar verzeichnet, sind Monitoring und Incident-Zuständigkeiten geregelt, und ist das Verhalten alter Instanzen nach dem Release verstanden? Ist das Verfahren für Rücknahme, Deaktivierung und gegebenenfalls destruktive Löschung dokumentiert?

Als Abschlussregel wird empfohlen: Ein technisch erfolgreicher Import erzeugt **noch keine fachliche Produktionsfreigabe**. Diese folgt aus der Verbindung eines identifizierten Release-Stands, freigegebener Zielkonfiguration, überprüfter Schnittstellen und einem verantwortlichen Entscheid. Welche Teile davon atlas künftig selbst als durchgängigen Workflow anbietet, bleibt einem gesonderten Architekturentscheid vorbehalten.